

## PRACTICAL- 8 (A)

### AIM

Machine Learning

a. To implement data pre-processing techniques including handling missing values, normalization/standardization, and feature encoding on a given dataset

### THEORY

#### 1. Data Preprocessing in Machine Learning

Data preprocessing is an important step in machine learning that involves **cleaning and transforming raw data into a suitable format** for building models. Real-world datasets often contain missing values, inconsistent formats, and categorical variables that need to be converted into numerical form.

Proper preprocessing improves:

- Model accuracy
- Training efficiency
- Reliability of predictions

#### 2. Handling Missing Values

Datasets may contain missing values due to:

- Data collection errors
- Incomplete records
- Data corruption

And other methods like normalization, standardization, outlier detection etc.

### SOFTWARE USED

Jupyter Notebook

### CODE

```
import pandas as pd

from sklearn.preprocessing import StandardScaler, MinMaxScaler

from sklearn.impute import SimpleImputer
```

```
# Sample dataset
```

```

data = {
    'Age': [25, 30, None, 40, 35],
    'Salary': [50000, 60000, 65000, None, 70000],
    'City': ['Delhi', 'Mumbai', 'Delhi', 'Pune', 'Mumbai']
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# Handling missing values
imputer = SimpleImputer(strategy='mean')
df[['Age', 'Salary']] = imputer.fit_transform(df[['Age', 'Salary']])

# Feature Encoding
df_encoded = pd.get_dummies(df, columns=['City'])

# Normalization
scaler = MinMaxScaler()
df_encoded[['Age', 'Salary']] = scaler.fit_transform(df_encoded[['Age', 'Salary']])

print("\nProcessed Dataset:")
print(df_encoded)

```

# OUTPUT

```
Original Dataset:
  Age  Salary  City
0  25.0  50000.0  Delhi
1  30.0  60000.0  Mumbai
2   NaN  65000.0  Delhi
3  40.0    NaN   Pune
4  35.0  70000.0  Mumbai

Processed Dataset:
  Age  Salary  City_Delhi  City_Mumbai  City_Pune
0  0.000000  0.0000      True         False      False
1  0.333333  0.5000      False         True       False
2  0.500000  0.7500      True         False      False
3  1.000000  0.5625      False         False      True
4  0.666667  1.0000      False         True       False
```

## **PRACTICAL- 8 (B)**

### **AIM**

Machine Learning

b. To build and evaluate a Linear Regression model for predicting continuous outcomes using an appropriate real-world dataset.

### **THEORY**

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. It models the relationship between an independent variable (input) and a dependent variable (output) by fitting a straight line to the data. The main objective of linear regression is to find the best-fitting line that minimizes the difference between the predicted values and the actual values. This line is represented by the equation  $y = \beta_0 + \beta_1 x$ , where  $y$  is the predicted output,  $x$  is the input variable,  $\beta_0$  is the intercept, and  $\beta_1$  is the slope of the line. Linear regression is widely used in real-world applications such as house price prediction, sales forecasting, demand prediction, and economic analysis. The performance of the model is commonly evaluated using metrics such as Mean Squared Error (MSE) and  $R^2$  score, which measure how well the model predicts the data.

### **SOFTWARE USED**

Jupyter Notebook

### **CODE**

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score


# Sample dataset

data = {

    'House_Size': [800, 1000, 1200, 1500, 1800, 2000],

    'Price': [150000, 200000, 250000, 300000, 350000, 400000]

}
```

```
df = pd.DataFrame(data)

X = df[['House_Size']]
y = df['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
predictions = model.predict(X_test)

print("Predicted Prices:", predictions)

# Evaluation
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

## OUTPUT

```
Predicted Prices: [164977.97356828 361013.21585903]
Mean Squared Error: 172815307.88488048
R2 Score: 0.982718469211512
```